

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T032	Name: KHUSHI
Class: TYIT	Batch: 2
Date of Assignment: 14-07-26	Date/Time of Submission: 14-07-26

A] Create methods add(), multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

ALGORITHM:-

Step 1 - Start.

Step 2 - Declare a delegate with two integer parameters.

Step 3 - Create methods:

Add(int a, int b)

Subtract(int a, int b)

Multiply(int a, int b)

Divide(int a, int b)

Step 4 - In the Main() method:

Create a delegate object.

Assign the Add() method to the delegate and call it.

Assign the Subtract() method to the delegate and call it.

Assign the Multiply() method to the delegate and call it.

Assign the Divide() method to the delegate and call it.

Step 5 - Display the results.

Step 6 - Stop.

CODE:-

```
using System;
delegate void Operation(int a, int b);
class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition: " + (a + b));
    }

    public void Subtract(int a, int b)
    {
        Console.WriteLine("Subtraction: " + (a - b));
    }

    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiplication: " + (a * b));
    }

    public void Divide(int a, int b)
    {
        Console.WriteLine("Division: " + (a / b));
    }
}
public class Program
{
    public static void Main(string[] args)
    {
```

```

    Calculator c = new Calculator();
    Operation obj;

    obj = c.Add;
    obj(20, 10);

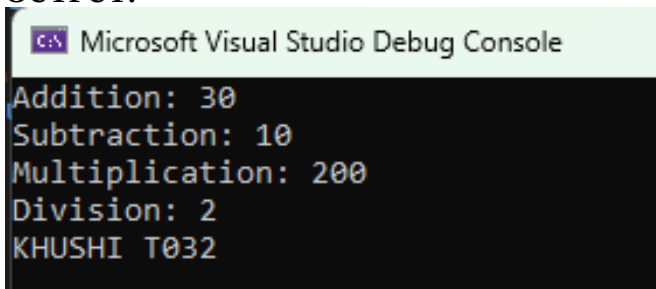
    obj = c.Subtract;
    obj(20, 10);

    obj = c.Multiply;
    obj(20, 10);

    obj = c.Divide;
    obj(20, 10);

    Console.WriteLine("KHUSHI T032");
}
}

```

OUTPUT:-


```

Microsoft Visual Studio Debug Console
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2
KHUSHI T032

```

B] Write a program using multicast delegate.**ALGORITHM:-****Step 1** - Start.**Step 2**- Declare a delegate with no parameters.**Step 3** - Create three methods (e.g., `Welcome()`, `Learn()`, `ThankYou()`).**Step 4** - In the `Main()` method:

Create a delegate object.

Assign the first method to the delegate.

Add the remaining methods using the `+=` operator.

Invoke the delegate object.

Step 5 - All methods execute one after another.**Step 6** - Stop.**CODE:-**

```

using System;
delegate void Message();
class Demo
{
    public void Welcome()
    {
        Console.WriteLine("Welcome Students");
    }
    public void learn()
    {
        Console.WriteLine("Leraning C# Delegates");
    }
}
public class Program
{
    public static void Main(string[] args)
    {
        Demo d = new Demo();
    }
}

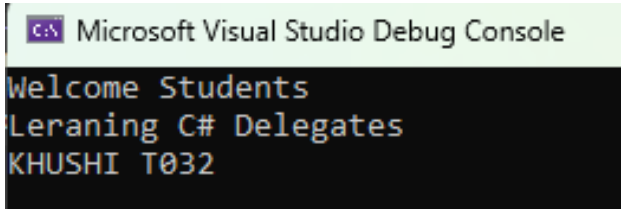
```

```

    Message obj;
    obj = d.Welcome;
    obj();
    obj = d.learn;
    obj();
    Console.WriteLine("KHUSHI T032");
}
}

```

OUTPUT:-



C] Create a class **BankAccount** with **AccountNumber** and **Balance**. Implement property for **Balance** with validation (Balance cannot be negative).

ALGORITHM:-

Step 1 - Start.

Step 2 - Create a class **BankAccount**.

Step 3 - Declare private data members:

AccountNumber

Balance

Step 4 - Create a property for **Balance**.

Step 5 - In the **set** accessor:

Check whether the assigned value is greater than or equal to 0.

If yes, store the value in **Balance**.

Otherwise, display an error message: **"Balance cannot be negative."**

Step 6 - Create an object of the **BankAccount** class in the **Main()** method.

Step 7 - Assign the account number.

Step 8 - Assign a valid balance and display the account details.

Step 9 - If a negative balance is entered, display the validation message.

Step 10 - Stop.

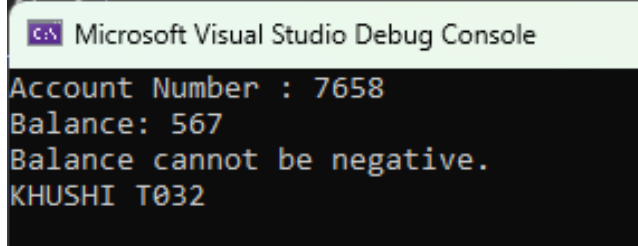
CODE:-

```

using System;
class BankAccount
{
    public int AccountNumber { get; set; }
    private double balance;
    public double Balance
    {
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
                balance = value;
            else
                Console.WriteLine("Balance cannot be negative.");
        }
    }
}
class Program

```

```
{  
    static void Main(string[] args)  
    {  
        BankAccount b = new BankAccount();  
        b.AccountNumber = 7658;  
        b.Balance = 567;  
        Console.WriteLine("Account Number : " + b.AccountNumber);  
        Console.WriteLine("Balance: " + b.Balance);  
        b.Balance = -500;  
        Console.WriteLine("KHUSHI T032");  
    }  
}
```

OUTPUT:-

Microsoft Visual Studio Debug Console

```
Account Number : 7658  
Balance: 567  
Balance cannot be negative.  
KHUSHI T032
```